

Sprawozdanie z Języków Formalnych i Technik Translacji

Zadanie 4: Konstrukcja pełnego analizatora LR(1)

Marcel Musiałek

Indeks: 279704

28 stycznia 2026

Spis treści

1	Treść Zadania	2
2	Opis Metody: Analiza LR(1)	2
2.1	Istota kanonicznego lookahead	2
2.2	Algorytm domknięcia (Closure)	2
3	Pełne Rozwiązanie: Kolekcja Kanoniczna Stanów	2
4	Pełna Tabela Analizatora LR(1)	3
5	Wnioski	3

1 Treść Zadania

Celem zadania jest policzenie pełnej tablicy kanonicznego analizatora LR(1) dla podanej gramatyki wyrażeń logicznych:

$$\begin{aligned}E &\rightarrow E \text{ or } T \mid T \\T &\rightarrow T \text{ and } F \mid F \\F &\rightarrow \text{not } F \mid (E) \mid \text{true} \mid \text{false}\end{aligned}$$

2 Opis Metody: Analiza LR(1)

2.1 Istota kanonicznego lookahead

Analizator LR(1) różni się od SLR tym, że każda sytuacja wewnątrz stanu (item) posiada przypisany własny zbiór symboli podglądu (*lookahead set*). Sytuacja ma postać $[A \rightarrow \alpha \cdot \beta, \{a, b\}]$, gdzie $\{a, b\}$ to terminale, po których wystąpieniu dopuszczalna jest redukcja do symbolu A .

2.2 Algorytm domknięcia (Closure)

Dla każdej sytuacji $[A \rightarrow \alpha \cdot B\beta, L]$ w stanie, dla każdej produkcji $B \rightarrow \gamma$, dodajemy sytuację:

$$[B \rightarrow \cdot \gamma, FIRST(\beta L)] \quad (1)$$

Dzięki temu lookahead jest precyzyjnie „dziedziczony” z kontekstu, co pozwala uniknąć konfliktów w gramatykach, w których zbiory *FOLLOW* są zbyt szerokie.

3 Pełne Rozwiązanie: Kolekcja Kanoniczna Stanów

Poniżej przedstawiono pełne stany automatu, z jawnym wyszczególnieniem zbiorów lookahead.

- **I0 (Stan początkowy):**
 $[E' \rightarrow \cdot E, \{\$ \}]$
 $[E \rightarrow \cdot E \text{ or } T, \{\$, \text{or}\}]$
 $[E \rightarrow \cdot T, \{\$, \text{or}\}]$
 $[T \rightarrow \cdot T \text{ and } F, \{\$, \text{or}, \text{and}\}]$
 $[T \rightarrow \cdot F, \{\$, \text{or}, \text{and}\}]$
 $[F \rightarrow \cdot \text{not } F, \{\$, \text{or}, \text{and}\}]$
 $[F \rightarrow \cdot (E), \{\$, \text{or}, \text{and}\}]$
 $[F \rightarrow \cdot \text{true}, \{\$, \text{or}, \text{and}\}]$
 $[F \rightarrow \cdot \text{false}, \{\$, \text{or}, \text{and}\}]$
- **I1 (GOTO(I0, E)):**
 $[E' \rightarrow E \cdot, \{\$ \}] \rightarrow \mathbf{ACCEPT}$
 $[E \rightarrow E \cdot \text{ or } T, \{\$, \text{or}\}]$
- **I2 (GOTO(I0, T)):**
 $[E \rightarrow T \cdot, \{\$, \text{or}\}] \rightarrow \mathbf{Redukcja (2)}$
 $[T \rightarrow T \cdot \text{ and } F, \{\$, \text{or}, \text{and}\}]$

- **I4** ($GOTO(I0, ' (')$):
 $[F \rightarrow (\cdot E), \{\$, or, and\}]$
 $[E \rightarrow \cdot E \text{ or } T, \{ \}, or\}]$
 $[E \rightarrow \cdot T, \{ \}, or\}]$
 $[T \rightarrow \cdot T \text{ and } F, \{ \}, or, and\}]$
 $[T \rightarrow \cdot F, \{ \}, or, and\}]$
(Zauważmy zmianę symbolu \$ na) w zbiorze podglądu).
 - **I10** ($GOTO(I3, F)$):
 $[F \rightarrow not \ F \cdot, \{\$, or, and\}] \rightarrow \text{Redukcja (5)}$
-

4 Pełna Tabela Analizatora LR(1)

Tabela zawiera 12 kolumn, uwzględniając wszystkie terminale (ACTION) i nieterminale (GOTO).

Stan	ACTION							GOTO			
	or	and	not	(	)	true	false	\$	E	T	F
0			s3	s4		s8	s9		1	2	5
1	s6							acc			
2	r2	s7						r2			
3			s3	s4		s8	s9				10
4			s14	s15		s25	s26		11	12	13
5	r4	r4						r4			
6			s3	s4		s8	s9			16	5
7			s3	s4		s8	s9				17
8	r7	r7						r7			
9	r8	r8						r8			
10	r5	r5						r5			
11	s19				s18						
12	r2	s20			r2						
13	r4	r4			r4						
14			s14	s15		s25	s26				21
15			s14	s15		s25	s26		11	12	13
16	r1	s7						r1			
17	r3	r3						r3			
18	r6	r6						r6			
19			s14	s15		s25	s26			23	13
20			s14	s15		s25	s26				24
21	r5	r5			r5						
22	r6	r6			r6						
23	r1	s20			r1						
24	r3	r3			r3						
25	r7	r7			r7						
26	r8	r8			r8						

5 Wnioski

Kanoniczny analizator LR(1) wyeliminował niejednoznaczności poprzez precyzyjne rozdzielanie stanów o tych samych rdzeniach, ale różnych kontekstach (np. stany I10 i I21). Jest to metoda najbardziej ogólna, choć kosztowna pod względem liczby stanów w porównaniu do metod SLR.